

2. Extração de Features — Flow Table Controller

2.1 Objetivo da entrega

Nesta primeira versão, o Controller deverá receber os metadados extraídos do pacote, solicitar a consulta da posição indicada pelo `hash_index`, verificar se a entrada corresponde a um fluxo existente ou a uma posição livre e, a partir dessa classificação, executar uma das três operações principais:

- **MISS:** criação de uma nova entrada na Flow Table;
- **HIT:** atualização de uma entrada de fluxo existente;
- **COLLISION:** identificação de uma posição ocupada por outro fluxo.

Além do processamento dos pacotes, o Controller também será responsável por coordenar a **finalização dos fluxos**, iniciada após o término da leitura do PCAP.

2.2 Entradas e informações utilizadas pelo Controller

O Controller recebe os metadados produzidos pelas etapas anteriores da arquitetura:

Sinal	Largura	Função
<code>in_valid</code>	1 bit	Indica que os metadados do pacote são válidos
<code>in_ready</code>	1 bit	Indica que o Controller está disponível para receber um novo pacote
<code>in_flow_key</code>	104 bits	Identificador do fluxo
<code>in_hash_index</code>	5 bits	Posição inicial da Flow Table obtida pelo hash
<code>in_packet_length</code>	16 bits	Tamanho do pacote
<code>in_timestamp</code>	64 bits	Timestamp associado ao pacote
<code>pcap_done</code>	1 bit	Indica o término do processamento do PCAP

A chave do fluxo possui **104 bits**, sendo composta por:

`src_ip + dst_ip + src_port + dst_port + protocol`

Para os primeiros testes, a Flow Table será considerada com **32 entradas**, portanto o índice utilizado para acesso possui **5 bits**.

2.3 Entregável 1 — Fluxograma do Controller

O fluxo de processamento proposto para o Controller é baseado na sequência:

Recepção → Lookup → Verificação → MISS/HIT/COLLISION → Operação correspondente

2.3.1 Início do processamento

O Controller permanece no estado de espera enquanto não houver um pacote válido.

Quando:

in_valid = 1 e in_ready = 1

ocorre a transferência dos metadados para o Controller.

Nesse momento, o Controller deve armazenar os metadados necessários para o processamento do pacote e utilizar o in_hash_index para solicitar a leitura da posição correspondente da Flow Table.

2.3.2 Lookup

O Controller solicita à Flow Table a leitura da entrada indicada pelo hash_index.

São obtidos:

- bit valid;
- flow_key armazenada;
- packet_count;
- byte_count;
- first_timestamp;
- last_timestamp.

A partir dessas informações, o Controller realiza a classificação da entrada.

2.3.3 Caminho de MISS

Ocorre quando:

valid = 0

Nesse caso, a posição da Flow Table está livre.

O Controller deve solicitar a criação de uma nova entrada contendo:

Campo	Valor inicial
valid	1
flow_key	in_flow_key
packet_count	1
byte_count	in_packet_length

Campo	Valor inicial
first_timestamp	in_timestamp
last_timestamp	in_timestamp

Após a escrita da entrada, o processamento daquele pacote é concluído e o Controller pode retornar ao estado de espera.

2.3.4 Caminho de HIT

Ocorre quando:

valid = 1

e

stored_flow_key = in_flow_key

Nesse caso, a entrada corresponde ao fluxo que já está armazenado na Flow Table.

O Controller encaminha os dados da entrada juntamente com os novos metadados do pacote para o **Feature Update Unit**.

A atualização realizada será:

packet_count = packet_count + 1

byte_count = byte_count + packet_length

first_timestamp = first_timestamp

last_timestamp = timestamp

Após a conclusão da atualização, o Controller retorna ao processamento de novos pacotes.

2.3.5 Caminho de COLLISION

Ocorre quando:

valid = 1

e

stored_flow_key $\neq$ in_flow_key

Nesse caso, a posição indicada pelo hash está ocupada, mas pertence a outro fluxo.

Para a primeira versão do projeto, será adotado um tratamento simplificado:

- o Controller identifica a colisão;
- não sobrescreve a entrada existente;
- sinaliza a ocorrência da colisão;

- o pacote não é inserido na Flow Table;
- o Controller retorna ao estado de espera.

2.3.6 Caminho de finalização

Quando pcap_done = 1, o Controller deixa de aceitar novos pacotes e inicia a etapa de finalização.

A Flow Table é percorrida sequencialmente, desde a primeira até a última posição.

Para cada entrada:

- se valid = 0, a posição é ignorada;
- se valid = 1, os dados são enviados ao **Flow Finalizer**.

O Finalizer calcula:

$\text{duration} = \text{last_timestamp} - \text{first_timestamp}$

e posteriormente gera o vetor final de features.

Após o processamento da última entrada válida, o Controller sinaliza que a finalização foi concluída.

2.4 Entregável 2 — Primeira versão da FSM

A primeira versão da máquina de estados finitos do Controller é composta pelos seguintes estados:

Estado	Função
IDLE	Aguarda a chegada de um novo pacote
LOOKUP	Solicita a leitura da posição da Flow Table
CHECK	Analisa a entrada retornada
MISS	Prepara a criação de uma nova entrada
INSERT	Escreve a nova entrada na Flow Table
HIT	Prepara a atualização de um fluxo existente
UPDATE	Aguarda a conclusão da atualização
COLLISION	Trata a colisão identificada
FINALIZE_SCAN	Percorre as posições da Flow Table
FINALIZE_SEND	Envia uma entrada válida ao Finalizer

Estado	Função
FINALIZE_WAIT	Aguarda o término do processamento pelo Finalizer
DONE	Indica que a finalização foi concluída

2.4.1 Transições da FSM

Estado atual	Condição	Próximo estado	Operação
IDLE	in_valid && in_ready	LOOKUP	Captura novo pacote
IDLE	pcap_done	FINALIZE_SCAN	Inicia finalização
LOOKUP	leitura concluída	CHECK	Recebe entrada
CHECK	valid = 0	MISS	Identifica posição livre
CHECK	valid = 1 e chave igual	HIT	Identifica fluxo existente
CHECK	valid = 1 e chave diferente	COLLISION	Identifica colisão
MISS	—	INSERT	Prepara nova entrada
INSERT	escrita concluída	IDLE	Conclui inserção
HIT	—	UPDATE	Solicita atualização
UPDATE	update_done	IDLE	Conclui atualização
COLLISION	tratamento concluído	IDLE	Retorna à espera
FINALIZE_SCAN	entrada inválida	FINALIZE_SCAN	Avança para próxima posição
FINALIZE_SCAN	entrada válida	FINALIZE_SEND	Prepara envio
FINALIZE_SEND	envio aceito	FINALIZE_WAIT	Aguarda Finalizer
FINALIZE_WAIT	finalizer_done	FINALIZE_SCAN	Continua varredura
FINALIZE_SCAN	última posição processada	DONE	Finaliza processamento
DONE	reinício autorizado	IDLE	Retorna ao processamento

Observação: a condição exata de retorno de DONE para IDLE ainda pode depender da definição da interface de controle do sistema.

2.5 Entregável 3 — Sequência das operações

A sequência geral de processamento de um pacote pode ser descrita da seguinte maneira.

Etapa 1 — Recepção

O Controller aguarda:

`in_valid && in_ready = 1`

Quando essa condição ocorre, os metadados do pacote são capturados.

Etapa 2 — Lookup

O `in_hash_index` é utilizado para selecionar a posição da Flow Table.

O Controller solicita a leitura dessa posição.

Etapa 3 — Classificação

Após a leitura, o Controller compara:

1. o bit valid;
2. a chave armazenada;
3. a chave recebida.

A classificação é:

`valid = 0`

→ MISS

`valid = 1` e chave igual

→ HIT

`valid = 1` e chave diferente

→ COLLISION

Etapa 4 A— MISS

Para um MISS, o Controller cria uma nova entrada.

`packet_count = 1`

`byte_count = packet_length`

`first_timestamp = timestamp`

`last_timestamp = timestamp`

A entrada é escrita na posição indicada pelo hash.

Etapa 4B — HIT

Para um HIT, a entrada existente é encaminhada ao Feature Update Unit.

O módulo realiza:

`packet_count = packet_count + 1`

`byte_count = byte_count + packet_length`

`last_timestamp = timestamp`

O `first_timestamp` permanece inalterado.

Após a atualização, os dados devem ser armazenados novamente na Flow Table.

Etapa 4C — COLLISION

Para uma colisão, a entrada existente permanece inalterada.

O Controller sinaliza a colisão e descarta o pacote para a primeira versão do PoC.

Etapa 5 — Próximo pacote

Depois de concluída a operação, o Controller libera novamente `in_ready`, permitindo que um novo pacote seja recebido.

Essa serialização evita que um novo pacote seja processado antes que a atualização anterior tenha sido concluída.

2.6 Controle de valid/ready e backpressure

A interface de entrada utiliza o protocolo:

`transfer = valid && ready`

O produtor mantém `valid` ativo enquanto os dados não forem aceitos.

O Controller utiliza `in_ready` para controlar quando está disponível para receber um novo pacote.

Durante operações como:

- leitura da Flow Table;
- classificação;
- escrita;
- atualização;
- finalização;

o Controller pode manter:

in_ready = 0

até que a operação atual seja concluída.

Essa abordagem evita que dois pacotes sejam processados simultaneamente sobre a mesma entrada sem que exista um mecanismo adequado de controle de dependências.

Por exemplo, sem esse controle, dois pacotes pertencentes ao mesmo fluxo poderiam ler simultaneamente o mesmo packet_count e posteriormente gravar valores incorretos.

2.7 Entregável 4 — Dependências com os demais módulos

O Controller não trabalha de maneira isolada. Seu funcionamento depende das interfaces e comportamentos definidos pelos demais blocos da arquitetura.

2.7.1 Dependência com Metadata Extraction

O Controller depende do bloco anterior para receber:

- flow_key;
- hash_index;
- packet_length;
- timestamp;
- valid.

Portanto, é necessário definir:

- quando esses sinais são considerados válidos;
- em qual ciclo o hash_index está disponível;
- qual representação será utilizada para o timestamp;
- se todos os metadados estarão disponíveis simultaneamente.

2.7.2 Dependência com Flow Table

É necessário definir a interface de leitura e escrita da Flow Table.

Uma proposta inicial é:

Sinal	Direção	Função
ft_lookup_req	Controller → Flow Table	Solicita leitura
ft_lookup_index	Controller → Flow Table	Índice da posição
ft_read_valid	Flow Table → Controller	Indica leitura concluída

Sinal	Direção	Função
ft_read_data	Flow Table → Controller	Registro lido
ft_write_req	Controller → Flow Table	Solicita escrita
ft_write_index	Controller → Flow Table	Posição de escrita
ft_write_data	Controller → Flow Table	Dados a serem escritos
ft_write_done	Flow Table → Controller	Indica conclusão

Ainda é necessário confirmar se a própria Flow Table realizará as operações de escrita ou se o Controller enviará diretamente os dados para ela.

2.7.3 Dependência com Feature Update Unit

No caminho de HIT, o Controller precisa encaminhar:

- índice da entrada;
- estado atual do fluxo;
- tamanho do novo pacote;
- timestamp do novo pacote.

O Feature Update Unit retorna uma indicação de conclusão:

update_done

A interface definitiva precisa ser definida em conjunto para evitar duplicação de responsabilidades entre Controller e Update Unit.

2.7.4 Dependência com Flow Finalizer

Na etapa de finalização, o Controller fornece ao Finalizer os registros válidos da Flow Table.

O Finalizer precisa receber, no mínimo:

- packet_count;
- byte_count;
- first_timestamp;
- last_timestamp;
- informações necessárias para identificar o fluxo, caso sejam mantidas no vetor final.

O Finalizer calcula:

$duration = last_timestamp - first_timestamp$

e gera o vetor de features.

A transferência pode utilizar novamente valid/ready:

finalizer_valid && finalizer_ready

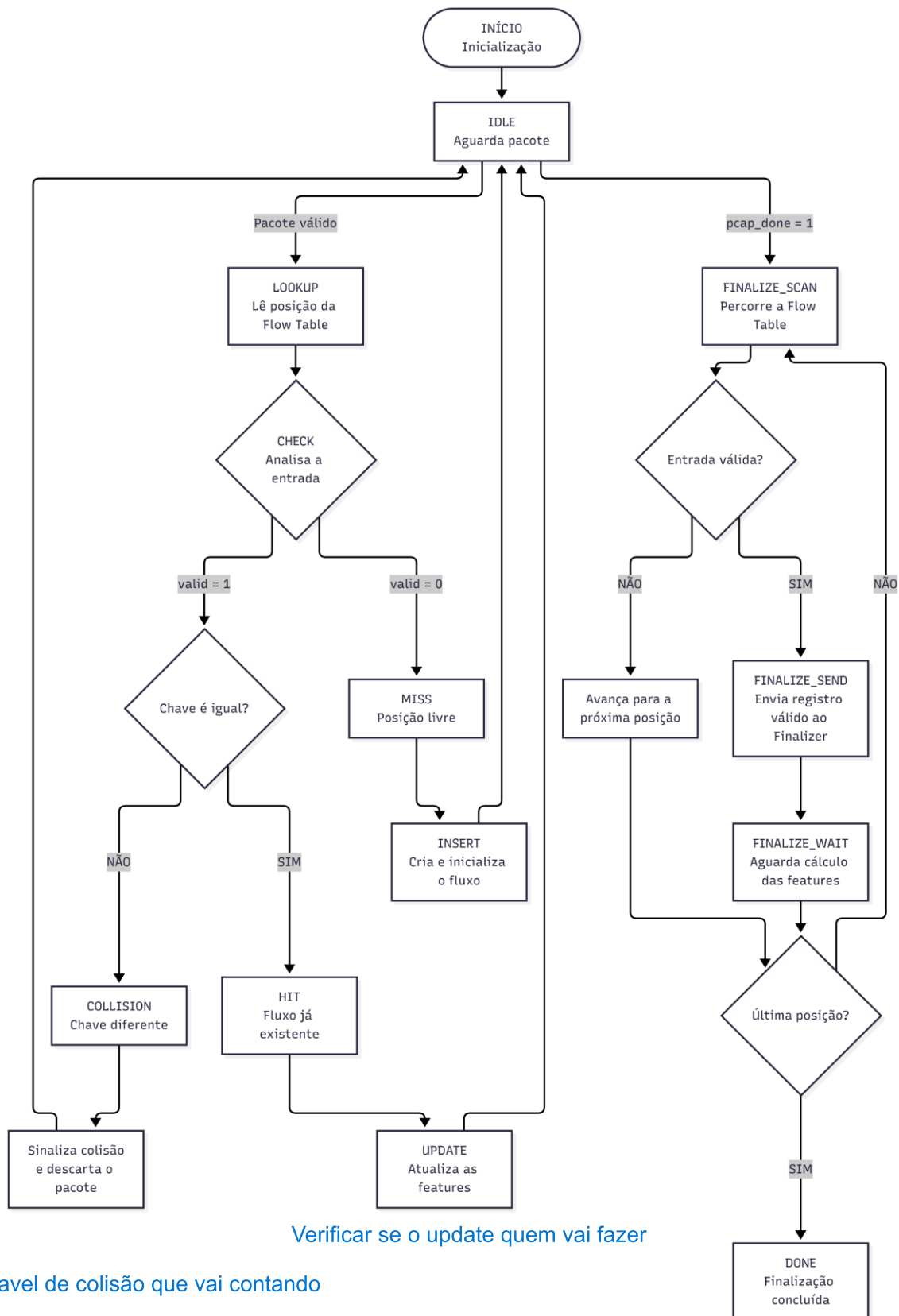
2.8 Pontos que precisam ser definidos

Antes do congelamento da interface do Controller, algumas decisões ainda precisam ser confirmadas.

Questão	Dependência
Qual será exatamente a interface da Flow Table?	Flow Table
A escrita da Flow Table será comandada pelo Controller?	Flow Table / Controller
Qual será a latência de leitura da Flow Table?	Flow Table
O hash_index será sempre válido quando in_valid = 1?	Hash Unit
Qual será a representação/unidade do timestamp?	Metadata / Verificação
O timestamp será fornecido pelo testbench?	Verificação
Qual será exatamente o formato do vetor final?	Finalizer / ML
O flow_key será incluído no vetor final ou somente usado para identificação?	Finalizer / ML
Como será sinalizada uma colisão?	Controller / Verificação
O que acontece com o processamento após DONE?	Sistema
Haverá posteriormente tratamento de timeout/FIN/RST?	Escopo futuro

2.9 Resumo da FSM

A lógica principal do Controller pode ser resumida em:



2.10 Resultado esperado da entrega

Ao final desta etapa, o **Flow Table Controller** terá uma primeira especificação funcional capaz de:

1. receber os metadados dos pacotes;
2. utilizar o hash_index para localizar uma posição na Flow Table;
3. identificar **MISS, HIT ou COLLISION**;
4. criar novas entradas em caso de MISS;
5. acionar a atualização de features em caso de HIT;
6. evitar a sobrescrita de entradas em caso de COLLISION;
7. controlar a recepção de novos pacotes por meio de valid/ready;
8. iniciar a finalização quando o PCAP terminar;
9. percorrer as entradas válidas da Flow Table;
10. encaminhá-las ao Flow Finalizer.

Os quatro entregáveis da sua parte ficam, portanto, claramente definidos:

Entregável	Resultado apresentado
1. Fluxograma	Fluxo completo de MISS, HIT, COLLISION e finalização
2. FSM	Estados, condições de transição e operações
3. Sequência das operações	Passo a passo do processamento de um pacote
4. Dependências	Interfaces e decisões que precisam ser alinhadas com os demais módulos

Importante: esta é uma **primeira especificação funcional**, não necessariamente a versão final da implementação RTL. As interfaces de memória, latências e alguns sinais ainda devem ser ajustados após a definição da Flow Table, Verificação e demais módulos.

Estrutura da flow table:

considerando **32 entradas**.

Cada entrada possui **297 bits**:

Campo	Largura	Função
valid	1 bit	Indica se a posição está ocupada
flow_key	104 bits	Identifica unicamente o fluxo
packet_count	24 bits	Quantidade de pacotes do fluxo

Campo	Largura	Função
byte_count	40 bits	Quantidade total de bytes do fluxo
first_timestamp	64 bits	Timestamp do primeiro pacote
last_timestamp	64 bits	Timestamp do pacote mais recente
Total	297 bits	Registro completo

Portanto:

Flow Table = 32×297 bits

ou aproximadamente:

9.504 bits $\approx$ 1.188 bytes

O flow_key de 104 bits

O flow_key é formado pela tupla:

src_ip

+

dst_ip

+

src_port

+

dst_port

+

protocol

Com:

Campo Bits

src_ip 32

dst_ip 32

src_port 16

dst_port 16

protocol 8

Total 104

Para início estamos considerando **uniflow**, então a inversão de origem/destino gera uma chave diferente.

Exemplo de uma entrada

Imagine que chegou um pacote com:

Flow Key = 104 bits

Packet Length = 1200 bytes

Timestamp = 5000

Hash Index = 12

Se a posição 12 estiver vazia:

valid = 1

flow_key = chave recebida

packet_count = 1

byte_count = 1200

first_timestamp = 5000

last_timestamp = 5000

Se posteriormente chegar outro pacote do **mesmo fluxo**, a entrada passa para algo como:

packet_count = 2

byte_count = 2400

first_timestamp = 5000

last_timestamp = novo timestamp

O first_timestamp **não muda** depois da criação do fluxo.

Um detalhe importante

O campo **duration não fica armazenado na Flow Table**.

Ele é calculado somente quando o fluxo é finalizado:

$duration = last_timestamp - first_timestamp$

Assim, a Flow Table mantém apenas o **estado necessário para a extração incremental**, e o Flow Finalizer calcula as features derivadas no final.

Essa é a estrutura que eu usaria como **estrutura preliminar do registro da Flow Table** na entrega desta semana.